

DBMS TASK 4 – Order Management System

Objective

To design a relational database for managing customer orders, order details, product quantities, order dates, total amounts, modifications, and customer order-history reports.

Database Design

The system uses Customers, Products, Orders, and Order_Details tables. Customers place orders. Each order can contain multiple products, while Order_Details stores the product quantity and price for each order.

Relationships

Customers (1) → Orders (Many)

Orders (1) → Order_Details (Many)

Products (1) → Order_Details (Many)

SQL – Create Tables

```
CREATE DATABASE OrderManagement;
USE OrderManagement;

CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    email VARCHAR(100)
);

CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL
);

CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

CREATE TABLE Order_Details (
    order_detail_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

Insert Sample Data

```
INSERT INTO Customers VALUES
(1, 'Arun', 'arun@gmail.com'),
(2, 'Priya', 'priya@gmail.com');

INSERT INTO Products VALUES
(101, 'Laptop', 55000.00),
(102, 'Mouse', 800.00),
(103, 'Keyboard', 1500.00);
INSERT INTO Orders VALUES
```

```
(1001, 1, '2026-08-17', 56600.00),
(1002, 2, '2026-08-17', 2300.00);
```

```
INSERT INTO Order_Details VALUES
(1, 1001, 101, 1, 55000.00),
(2, 1001, 102, 2, 800.00),
(3, 1002, 103, 1, 1500.00),
(4, 1002, 102, 1, 800.00);
```

Order Insertion

```
INSERT INTO Orders
(order_id, customer_id, order_date, total_amount)
VALUES (1003, 1, '2026-08-17', 1500.00);

INSERT INTO Order_Details
(order_detail_id, order_id, product_id, quantity, unit_price)
VALUES (5, 1003, 103, 1, 1500.00);
```

Order Modification

```
UPDATE Order_Details
SET quantity = 2
WHERE order_detail_id = 5;

UPDATE Orders
SET total_amount = 3000.00
WHERE order_id = 1003;
```

Customer Order History Report

```
SELECT c.customer_name,
       o.order_id,
       o.order_date,
       p.product_name,
       od.quantity,
       od.unit_price,
       o.total_amount
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Order_Details od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
ORDER BY c.customer_id, o.order_date;
```

Sample Report

Customer	Order ID	Product	Quantity	Total
Arun	1001	Laptop	1	56600.00
Arun	1001	Mouse	2	56600.00
Priya	1002	Keyboard	1	2300.00
Priya	1002	Mouse	1	2300.00

Conclusion

The Order Management System provides a structured relational database for storing customers, products, orders, and order details. Primary and foreign keys maintain data integrity, while SQL insertion, update, and reporting queries support daily order-management operations.